

作业感受（第二次挑战）

学号 0102603133

（1）简述下你是如何完成第二次挑战的（如何使用 AI、如何工作迭代等）

第二次挑战我分两步做。测量这步先把参数捋准：两组各 32 条、并发 8、temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026，两组序列等长（1024 前缀 + 128 后缀），每轮先 flush 缓存、共享前缀组再单独预热一条把公共前缀载入 RadixCache。跑出来的对比很直观：命中率 89.23% vs 0%，实际 prefill 3 969 vs 36 864 tokens，TTFT p50 差了一倍多，但 TPOT 只从小幅变化——这让我第一次把「缓存只影响 prefill 路径」这件事和数据对上了。读源码这步我沿 TokenizerManager → Scheduler → PrefillAdder → RadixCache → 模型前向 → Detokenizer 的主线，在本机装的 0.5.14 里逐个函数 grep 定位，画了一张带文件和行号的流程图，并把等待队列、前缀匹配削减 prefill、KV 写回、流式返回四个问题各写了一段。中间踩的坑主要在 API 假设上，比如 /flush_cache 其实返回纯文本，靠报错后才修正。

（2）你觉得哪部分工作最困难，你是如何克服这个困难的

比较难的是把“缓存真的命中了”这件事测准。第一版跑出来共享前缀组的命中率忽高忽低，我排查了挺久才发现两个问题：一是上一组请求还没跑完我就 flush 了缓存；二是 SGLang 的 /flush_cache 返回的是纯文本 "Cache flushed" 而不是 JSON，我按 JSON 解析，校验一直判失败，等于缓存压根没清干净就开始下一组了。改法是在 flush 之前先轮询等到没有在飞请求，再确认 flush 真的成功，而且共享前缀组还要单独先发一条预热请求把公共前缀推进 RadixCache，这条不计入结果。还有一个容易忽略的点：两组序列必须等长，不然 prefill 的差异说不清是来自缓存还是来自长度。我把两组都固定成 1024 前缀加 128 后缀，最后看到的 3 969 对 36 864 才是干净的对比。

读源码那一步也不轻松。SGLang 的目录很深，请求要跨 HTTP 进程、Scheduler 进程和 detokenizer 三处，光靠猜很容易画错流程。我的办法是顺着文档给的那条主线，用 grep 在本机装的 0.5.14 里逐个函数定位，读到哪标到哪，最后流程图上每个“文件：行号”都是实测过的。中途还发现文档里写的 cache_finished_req 在源码里其实是包了一层的——chunked 的走 maybe_cache_unfinished_req，完成的走 release_kv_cache，两者才真正调到 RadixCache 的写回入口，这个如果不看源码就会写错。

（3）类似的从 0 到 1 的科研工作需要包含哪些方面的研究，对你今后的科研工作有什么启发

这关给我最大的体会是控制变量比想象中难。测缓存效果听起来很简单，发两组请求比一下就行，真做的时候要管的变量有一堆：序列等长、并发相同、采样参数一致（temperature=0、max_new_tokens=16、ignore_eos、seed=2026）、每组开始时缓存状态一致、预热请求不能混进统计。漏掉任何一个，数据就没法解释。

第二点是得先想清楚指标量的是哪一段。我一开始以为命中率高就应该所有延迟都大幅下降，结果 TTFT p50 降了 55.8%，TPOT 只从 0.00537 降到 0.00317，绝对值本来就很小。因为前者是 prefill 主导、后者是 decode 主导，而缓存只作用在 prefill 那一段。读源码这个判断也得到印证：RadixCache.match_prefix 只影响 extend_input_len，之后每步 decode 该算多少还是多少。以后做实验我大概会先把“这个指标到底量的是哪一段”想清楚再动手跑数据，不然很容易拿一个指标的变化去解释另一个指标。